

Kosi hitac različitih projektila s otporom zraka

Katarina Filipović

Split, 15.06.2026.

Sažetak

Cilj mogega rada bio je uz pomoć programa u programskom jeziku *python* usporediti kako se za iste početne parametre giba kugla, a kako kocka uključujući i otpor zraka. Također, program ima funkciju računanja svih mogućih kuteva pod kojima bi projektil pogodio metu za zadane početne uvjete.

1 Uvod

Kosi hitac je gibanje tijela u homogenom gravitacijskom polju pod nekim kutom u odnosu na horizontalu. Znamo da se ono sastoji od dva neovisna gibanja; jednolikog gibanja po pravcu (x -os) i jednoliko ubrzanog/usporenog gibanja (y -os).

Iako se otpor zraka vrlo često zanemaruje, moramo biti svjesni kako on u stvarnome svijetu može imati znatan utjecaj na rezultate, zato u ovome radu proučavam kosi hitac s otporom zraka, gdje na tijelo osim sile teže djeluje i sila otpora zraka, u ovom slučaju proporcionalna kvadratu brzine ($F \sim v^2$). Formula korištena za računanje otpora zraka je

$$F_t = kv^2, \quad k = \frac{1}{2}C_d\rho A$$

gdje je C_d koeficijent koji ovisi o obliku. Za kuglu je on $C_d = 0.47$, a za kocku $C_d = 1.05$. ρ je gustoća zraka, A je poprečni presjek projektila okomit na smjer gibanja, a v trenutna brzina.

Primjenom II. Newtonovog zakona ($\vec{F} = m\vec{a}$) dobivaju se povezane diferencijalne jednadžbe drugog reda koje se ne mogu riješiti analitički zbog člana v^2 , što je glavni razlog zašto moramo koristiti računalo i numeričke metode. Ja sam za svoj rad odabrala Runge-Kutte metodu 4. reda iz razloga što je mnogo preciznija od klasične Eulerove koja s vremenom akumulira pogrešku. Runge-kutta u svakom koraku računa četiri različita nagiba (k_1, k_2, k_3, k_4) i uzima njihovu sredinu. Zato je iznimno precizna i pogodnija za korištenje u ovome slučaju. Formule su preuzete sa [1]

2 Diskusija i rezultati

```
class Projectile:
```

```

def __init__(self, v, k, m, x, y, rho=1.225, oblik='kugla', ra=0.06):
    (...)
    self.x_r4= [self.x]
    self.y_r4= [self.y]

```

Koristeći klasu *Projectile* definirane su sve varijable potrebne za simulaciju kosog hitca. Također, napravljene su dvije liste u koje će se spremati koordinate putanje po kojoj se projektil giba. Nadalje, treba naglasiti da su projektili u ovom istraživanju nerotirajući, tako da je koeficijent otpora zraka k promjenjiv u vremenu. Dodana je nova unutarnja metoda koja ga računa.

```

def __b(self, vx, vy):
    if self.oblik == "kugla":
        A = (self.ra**2) * math.pi
    elif self.oblik == "kocka":
        if vx!= 0:
            theta= math.atan2(vy, vx)
        else:
            theta= math.pi / 2 #vertikalnogibanje
        A= (self.ra**2)* (abs(math.cos(theta)) + abs(math.sin(theta)))
    return (0.5 * self.rho * self.c * A)

```

U slučaju da je projektil kugla, njegov koeficijent otpora zraka je stalan zato što kugla ne mijenja svoj poprečni presjek gledano iz bilo kojeg kuta. Međutim, kod kocke to nije slučaj zato što se površina kocke okomita na smjer gibanja mijenja. Površina presjeka dana je formulom $A = a^2 \cdot (|\cos \theta| + |\sin \theta|)$ koja ustvari računa projekciju (nerotirajuće) kocke na ravninu okomitu na smjer gibanja.

Nakon što je koeficijent otpora zraka b definiran primijenjena je Runge-Kutta metoda 4. reda. U nastavku je prikazan primjer računanja prvog koeficijenta nagiba k_1 . Poanta ove metode je da se svaki sljedeći nagib računa pomoću brzine dobivene u prethodnom koraku. Kada su određena sva četiri nagiba, novi položaj i brzina računaju se pomoću njihove ponderirane srednje vrijednosti. Pritom se veća težina pridaje koeficijentima 2 i 3 jer su računati u polukoraku ($\frac{dt}{2}$), što metodi osigurava visoku preciznost.

```

def __R4(self, dt):
    g=9.81
    #k1
    v1=np.sqrt(self.vx**2+self.vy**2)
    b1=self.__b(self.vx, self.vy)
    ax1=-(b1/self.m)*v1*self.vx
    ay1=-g-(b1/self.m)*v1*self.vy

    (...)

    self.x=self.x+(dt/6)*(self.vx+2*vx2+2*vx3+vx4)
    self.y=self.y+(dt/6)*(self.vy+2*vy2+2*vy3+vy4)
    self.vx=self.vx+(dt/6)*(ax1+2*ax2+2*ax3+ax4)

```

```

self.vy=self.vy+(dt/6)*(ay1+2*ay2+2*ay3+ay4)

self.x_r4.append(self.x)
self.y_r4.append(self.y)

```

Sada je moguće pomicati projektil. Za tu je svrhu stvorena metoda *graf*:

```

def graf(self):
    dt=0.0001
    while self.y >= 0:
        self.__R4(dt)
    return self.x_r4, self.y_r4

```

Potom je implementirana nova metoda koja programu dodaje funkciju pronalaska kutova pod kojima će projektil pogoditi metu. Provjeravat će se poklapaju li se barem jednom koordinate na putanji projektila sa koordinatama mete za svaki kut do 90°, gdje je korak 0.1°. S obzirom da ni projektil ni meta nisu samo točke u prostoru, postavljen je uvjet da je meta pogođena ako se projektil nalazi u točki čija je udaljenost od koordinata mete manja ili jednaka radijusu mete. U kodu je opet primijenjena R4 metoda.

```

def pronadji_kut(self, xM, yM, rM):
    kutovi_pogotka= []
    dt= 0.001
    (...)

    kutovi= list(np.arange(0.5, 90.0, 0.1))
    for kut in kutovi:
        radijani = math.radians(kut)
        (...)

    pogodak = False
    for i in range(len(testni_x)):
        d = math.sqrt((testni_x[i] - xM)**2 + (testni_y[i] - yM)**2)
        if d <= rM:
            pogodak = True
            kutovi_pogotka.append(round(float(kut), 1))
            break

```

Ovime su dovršene metode potrebne za pokretanje koda. Sada je klasa spremna za korištenje u zadatku pomoću funkcije "import" na sljedeći način:

```

from tema11 import Projectile

```

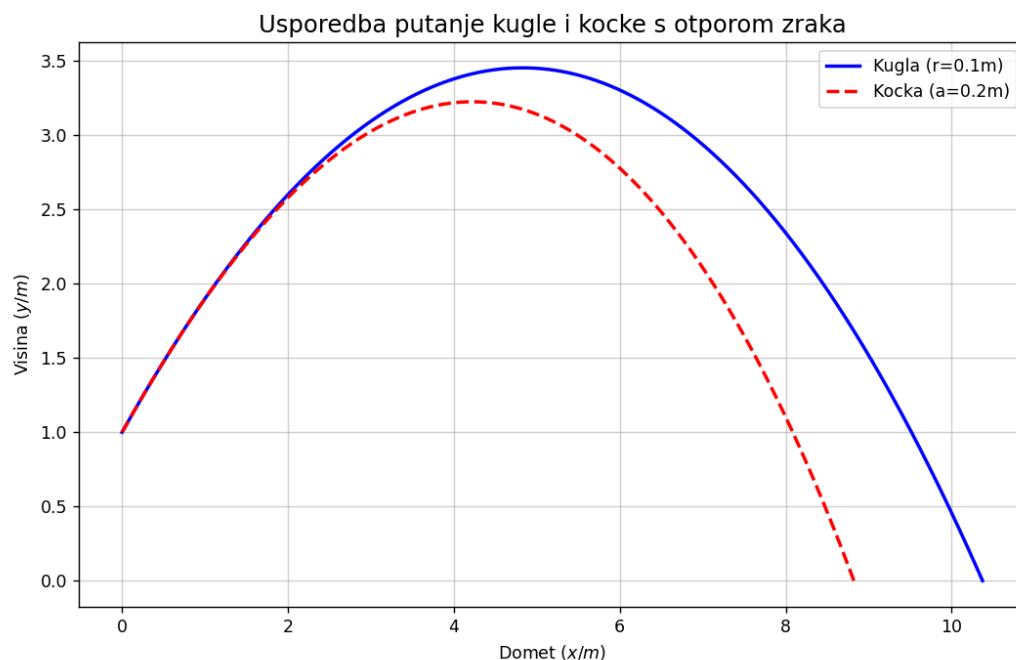
Nakon toga su stvoreni objekti pomoću uvezene klase;

```

kugla = Projectile(v, kut, m, x0, y0, oblik="kugla", ra=0.1)
kocka = Projectile(v, kut, m, x0, y0, oblik="kocka", ra=0.2)

```

Nadalje, korištenjem python biblioteke *matplotlib.pyplot* nacrtan je graf s kojeg je moguće očitati rezultate. (slika 1)



Slika 1: Putanja kocke i kugle

Vrlo se lako može uočiti da je na grafu dobiven očekivani rezultat, tj. da je domet kocke manji za iste početne uvjete (kugla je takve veličine da se može upisati u kocku, u primjeru je radijus $r=0.1\text{m}$, a duljina brida kocke $a=0.2\text{m}$).

Sljedeći zadatak ovog istraživanja bio je odrediti sve kutove pod kojima bi projektil za neke početne uvjete pogodio metu oblika kugle određenih koordinata i radijusa.

```
projektil = Projectile(v, k, m, x, y, oblik='kugla', ra=0.1)
lista_kuteva= projektil.pronadji_kut(xM, yM, rM)
```

Definiran je projektil za neku metu. Pri pronalasku kutova dobiven je veliki broj sličnih kutova zbog toga što i projektil i meta imaju određene dimenzije, a nisu samo točke u prostoru. Za prikaz na grafu dovoljno će biti uzeti samo po jedan kut od bliskih kutova. S obzirom da su moguće tri opcije za rješenje (kut za nisku i visoku putanju, samo jedan kut, nijedan kut), kroz petlju su provjerene razlike između kutova tako da možemo odrediti radi li se o dvije, jednoj ili nijednoj putanji. Kutovi su dodani u listu s manjim kutovima ako je razlika između njih manja od 0.5, a ako je veća dodani su u listu s većim kutovima.

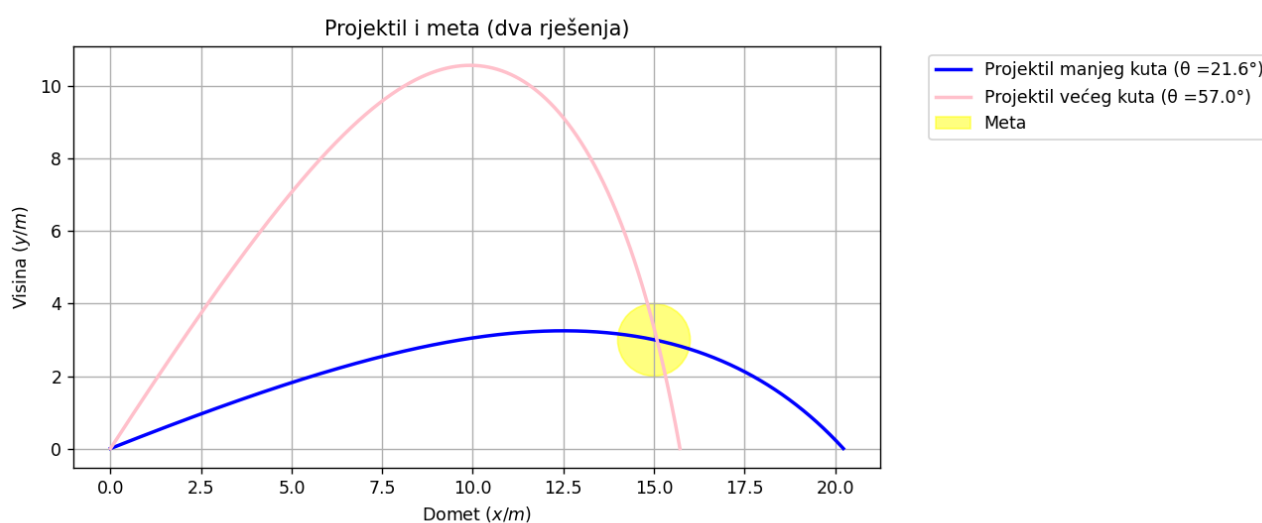
```
niska_putanja = []
visoka_putanja = []
```

```

if len(lista_kuteva) > 0:
    niska_putanja.append(lista_kuteva[0])
    for k_tren in lista_kuteva[1:]:
        if k_tren - niska_putanja[-1] > 0.5:
            visoka_putanja.append(round(float(k_tren), 1))
        else:
            niska_putanja.append(round(float(k_tren), 1))

```

Korištenjem if petlje ispitan je broj rješenja i granice mogućih kutova, a potom je i graf nacrtan korištenjem iste petlje. Na slici 2 su prikazani dobiveni rezultati za projektil koji se giba iz ishodišta, početnom brzinom $v = 35\text{m/s}$, mase $m = 0.1\text{kg}$, oblika kocke brida $r = 12\text{cm}$ i standardne gustoće zraka iznad mora pri temperaturi od 15°C . Meta ima koordinate $(15,3)$ u m , a radijus joj je $r = 1\text{m}$.



Slika 2: Projektil i meta

3 Zaključak

Na kraju možemo zaključiti kako je otpor zraka vrlo važan i nezanemariv faktor pri kretanju nekog objekta.

Nadalje, rad demonstrira kako različita geometrijska svojstva tijela također izravno utječu na gibanje tijela.

Isto tako možemo zaključiti kako je bez adekvatne računalne opreme vrlo teško ručno analitički riješiti problem i tu nam je programiranje od velike pomoći.

Literatura

- [1] Mohd Faraz, Aachen University: https://www.researchgate.net/publication/329339455_Study_of_projectile_motion_with_air_resistance